

PostgreSQL 16

Parameter Tuning for Large Restores

ZERO DOWNTIME — RELOAD ONLY

All parameters in this runbook are reload-safe. No restarts required. No standby disconnection. No replication risk.

Item	Detail
PostgreSQL Version	16
Replication Type	Streaming Replication (with replication slot)
Replication Slot	ssncpri
Data Directory	/apps/pgsql_data/16
Restart Required?	NO — pg_reload_conf() only

Why no wal_buffers? The default auto-tuned value (~64 MB) is already sufficient for heavy restores. Changing wal_buffers requires a full restart — which introduces replication risk. Skipping it means every parameter change can be applied and reverted with just a reload. The speed gain from wal_buffers = 256 MB is negligible compared to the risk it introduces.

SECTION 1: Parameter Values

Parameter	Default	Restore Value	Impact on Restore Speed
<code>maintenance_work_mem</code>	64MB	2GB	MASSIVE — speeds up CREATE INDEX, FK validation
<code>max_wal_size</code>	1GB	20GB	HIGH — reduces checkpoint frequency, fewer I/O stalls
<code>checkpoint_timeout</code>	5min	30min	HIGH — fewer checkpoints during bulk load
<code>work_mem</code>	4MB	256MB	MEDIUM — faster sorts/joins during FK creation
<code>min_wal_size</code>	80MB	5GB	LOW — keeps WAL pre-allocated
<code>checkpoint_completion_target</code>	0.9	0.9	Keep at 0.9 — DO NOT lower
<code>wal_keep_size</code>	0	50GB	No speed impact — PROTECTS REPLICATION
<code>max_slot_wal_keep_size</code>	-1	-1	No speed impact — PROTECTS SLOT

All parameters above are **reload-safe** — they take effect with `SELECT pg_reload_conf();` without any restart.

The two CRITICAL safety parameters:

`wal_keep_size = '50GB'` — Keeps old WAL on disk even after checkpoints. Safety net if standby falls behind.

`max_slot_wal_keep_size = '-1'` — Never forcibly remove WAL that the replication slot needs. If this is set to a finite value, PostgreSQL will INVALIDATE the slot.

SECTION 2: Pre-Change Verification

Do this FIRST. Verify replication is healthy before changing anything.

Run on PRIMARY:

SQL — Check Replication Status

```
-- 1. Check replication status
SELECT pid, username, client_addr, state, sent_lsn, write_lsn,
       flush_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
-- state must be: streaming
```

SQL — Check Slot ssncpri

```
-- 2. Check replication slot health
SELECT slot_name, slot_type, active, restart_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
-- active must be: t
```

SQL — Record Current Values

```
-- 3. Record current parameter values (save this output)
SHOW max_wal_size;
SHOW min_wal_size;
SHOW checkpoint_timeout;
SHOW checkpoint_completion_target;
SHOW work_mem;
SHOW maintenance_work_mem;
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;
```

Run on STANDBY:

SQL — Verify Standby Status

```
SELECT pg_is_in_recovery();           -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming
```

DO NOT proceed if replication is not healthy.

SECTION 3: Apply Parameter Changes

Reload Order: STANDBY FIRST, then PRIMARY

Same principle as restarts — prepare the WAL consumer before increasing load on the producer.

Step 3a: Apply and Reload on STANDBY

SQL — ALTER SYSTEM + Reload on Standby

```
-- Connect to STANDBY via psql and run:

ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';

-- Apply immediately — NO RESTART NEEDED:
SELECT pg_reload_conf();
```

Verify changes took effect on standby:

SQL — Verify Standby After Reload

```
-- On STANDBY: verify new values are active
SHOW max_wal_size;           -- Should show: 20GB
SHOW maintenance_work_mem;   -- Should show: 2GB
SHOW wal_keep_size;          -- Should show: 50GB

-- Verify replication is still healthy (not interrupted):
SELECT pg_is_in_recovery();   -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming
```

Standby stays up the entire time. Replication never disconnects. Zero downtime.

Step 3b: Apply and Reload on PRIMARY

SQL — ALTER SYSTEM + Reload on Primary

```
-- Connect to PRIMARY via psql and run:

ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';

-- Apply immediately – NO RESTART NEEDED:
SELECT pg_reload_conf();
```

Verify changes took effect on primary:

SQL — Verify Primary After Reload

```
-- On PRIMARY: verify new values are active
SHOW max_wal_size;           -- Should show: 20GB
SHOW maintenance_work_mem;   -- Should show: 2GB
SHOW wal_keep_size;          -- Should show: 50GB

-- Verify replication is still flowing:
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots WHERE slot_name = 'ssncpri';
-- active must still be: t

SELECT client_addr, state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
-- state must still be: streaming
```

Both servers are now tuned for the restore. Replication was never interrupted. Slot ssncpri remained active throughout.

SECTION 4: Monitor During Restore

Monitor replication every 15-30 minutes during the restore.

SQL — Monitor Slot Lag

```
-- Run on PRIMARY periodically:
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
```

SQL — WAL Disk Usage

```
-- Check WAL disk usage:
SELECT pg_size_pretty(sum(size)) AS total_wal_on_disk
FROM pg_ls_waldir();
```

BASH — Disk Space

```
# Check disk space:
df -h /apps/pgsql_data/16/pg_wal
```

Warning Signs:

- **active = f** — Standby disconnected, investigate immediately
- **slot_lag growing rapidly** — Standby falling behind, but safe with wal_keep_size protecting WAL
- **WAL directory too large** — May need to pause restore

SECTION 5: Revert Parameters After Restore Completes

After restore is complete, FK creation is done, and everything is verified.

Step 5a: Revert and Reload on STANDBY FIRST

SQL — Individual Resets + Reload on Standby

```
-- On STANDBY:
-- NEVER use ALTER SYSTEM RESET ALL here – it will wipe
-- primary_conninfo from postgresql.auto.conf and break replication!

-- Reset ONLY the restore-tuning parameters individually:
ALTER SYSTEM RESET max_wal_size;
ALTER SYSTEM RESET min_wal_size;
ALTER SYSTEM RESET checkpoint_timeout;
ALTER SYSTEM RESET checkpoint_completion_target;
ALTER SYSTEM RESET work_mem;
ALTER SYSTEM RESET maintenance_work_mem;
ALTER SYSTEM RESET wal_keep_size;
ALTER SYSTEM RESET max_slot_wal_keep_size;

-- Apply immediately – NO RESTART NEEDED:
SELECT pg_reload_conf();
```

NEVER use ALTER SYSTEM RESET ALL on the standby. Your `primary_conninfo` lives in `postgresql.auto.conf` on the standby. RESET ALL will wipe it, and the next restart will break replication permanently. Always use individual RESET commands on the standby.

Verify standby after revert:

SQL — Verify Standby After Revert

```
-- On STANDBY:
SHOW max_wal_size;           -- Should be back to original
SHOW maintenance_work_mem;   -- Should be back to original

-- Replication still healthy:
SELECT pg_is_in_recovery();   -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming
```

Step 5b: Revert and Reload on PRIMARY SECOND

SQL — Reset + Reload on Primary

```
-- On PRIMARY:
ALTER SYSTEM RESET ALL;

-- Apply immediately – NO RESTART NEEDED:
SELECT pg_reload_conf();
```

Verify primary after revert:

SQL — Full Verification After Revert

```
-- On PRIMARY: parameters back to original
SHOW max_wal_size;
SHOW min_wal_size;
SHOW checkpoint_timeout;
SHOW work_mem;
SHOW maintenance_work_mem;
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;

-- Replication still healthy:
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots WHERE slot_name = 'ssncpri';

SELECT client_addr, state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
```

Verify postgresql.auto.conf is clean:

BASH — Verify Auto.conf on Both Servers

```
# On PRIMARY:
cat /apps/pgsql_data/16/postgresql.auto.conf
# Should be empty (RESET ALL cleared everything)

# On STANDBY:
cat /apps/pgsql_data/16/postgresql.auto.conf
# Should ONLY contain primary_conninfo (and possibly primary_slot_name)
# All restore-tuning parameters should be gone
```

REVERT COMPLETE. Both servers are back to original values. Replication was never interrupted throughout the entire process — from applying changes, through the restore, to reverting.

SECTION 6: Complete Summary

Reload Order — Both Phases

Phase	First	Second	Method
Applying changes	STANDBY	PRIMARY	pg_reload_conf() — no restart
Reverting changes	STANDBY	PRIMARY	pg_reload_conf() — no restart

Quick Reference Commands

Apply (run on both servers — standby first, primary second):

SQL — Apply (Copy-Paste Ready)

```
ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';

SELECT pg_reload_conf();
```

Revert (standby first, primary second):

SQL — Revert (Copy-Paste Ready)

```
-- STANDBY FIRST: individual resets only (protects primary_conninfo)
ALTER SYSTEM RESET max_wal_size;
ALTER SYSTEM RESET min_wal_size;
ALTER SYSTEM RESET checkpoint_timeout;
ALTER SYSTEM RESET checkpoint_completion_target;
ALTER SYSTEM RESET work_mem;
ALTER SYSTEM RESET maintenance_work_mem;
ALTER SYSTEM RESET wal_keep_size;
ALTER SYSTEM RESET max_slot_wal_keep_size;
SELECT pg_reload_conf();

-- PRIMARY SECOND: safe to use RESET ALL
ALTER SYSTEM RESET ALL;
SELECT pg_reload_conf();
```

Key Advantages of This Approach

- **Zero downtime** — no restarts on either server
- **Zero replication risk** — slot ssncri stays active throughout
- **Instant apply** — `pg_reload_conf()` takes effect in seconds
- **Instant revert** — `ALTER SYSTEM RESET + pg_reload_conf()`
- **No standby disconnection window** — eliminates the root cause of your previous failure
- **postgresql.conf untouched** — all changes live in `postgresql.auto.conf` only
- **NEVER use RESET ALL on standby** — always individual resets to protect `primary_conninfo`